



Université de Nouakchott

Faculté des Sciences et Techniques

Département de Mathématiques et Informatiques

Rapport de Projet

Optimisation pour la science de données

Étude et Implémentation d'Algorithmes d'Optimisation pour Données Massives

Réalisé par :

Boudah Mohamed Lemine Ahmedou El Mokhtar

C20121

Encadré par :

Dr.Mohamed Mahmoud El Bennany

Année universitaire : 2025 – 2026

Projet réalisé dans le cadre du Master SSD – Statistiques et Sciences des Données

Table des matières

1	Modélisation et Étude Théorique	2
1.1	Modélisation du problème	2
1.2	Étude du gradient et de la Hessienne	2
1.3	Lipschitz du gradient et Décomposition en Valeurs Singulières	3
2	Stochasticité et Passage à l'Échelle	4
2.1	Implémentation de la Descente de Gradient Stochastique (SGD)	4
2.2	Analyse Comparative : Batch Gradient vs SGD	4
2.3	Mini-batch et Adam	6
3	Parcimonie et Algorithmes Proximaux	8
3.1	Analyse géométrique de la régularisation L1	8
3.2	Algorithme ISTA	8
3.3	Accélération	9
	Conclusion	11

Exercice 1

Modélisation et Étude Théorique

1.1 Modélisation du problème

Soit $X \in \mathbb{R}^{n \times d}$ la matrice des données et $y \in \mathbb{R}^n$ le vecteur cible. Nous cherchons à prédire l'année de sortie d'une chanson en ajustant un vecteur de paramètres $w \in \mathbb{R}^d$.

Le problème de régression linéaire peut être formulé comme la minimisation de la fonction de coût quadratique moyenne :

$$f(w) = \frac{1}{n} \sum_{i=1}^n \ell_i(w), \quad \text{où} \quad \ell_i(w) = \frac{1}{2} (x_i^\top w - y_i)^2.$$

Pour garantir l'unicité du minimum global, on ajoute une pénalité de type ridge (L2) :

$$f_\mu(w) = \frac{1}{n} \sum_{i=1}^n \ell_i(w) + \frac{\mu}{2} \|w\|^2, \quad \mu > 0.$$

Justification : La fonction $f_\mu(w)$ est **fortement convexe** grâce à la pénalité L2. Selon le Théorème 1.2.9 du cours :

Si $f : \mathbb{R}^d \rightarrow \mathbb{R}$ est fortement convexe, elle possède un unique minimum global. De plus, si f est de classe C^1 , ce minimum est l'unique solution de $\nabla f(w) = 0$.

Ainsi, l'ajout du terme $\frac{\mu}{2} \|w\|^2$ garantit que le problème admet un **unique minimum global** w^* , solution de $\nabla f_\mu(w^*) = 0$.

1.2 Étude du gradient et de la Hessienne

La fonction de coût régularisée s'écrit :

$$f_\mu(w) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (x_i^\top w - y_i)^2 + \frac{\mu}{2} \|w\|^2.$$

Gradient

Le gradient est donné par :

$$\begin{aligned}\nabla f_\mu(w) &= \frac{1}{n} \sum_{i=1}^n x_i(x_i^\top w - y_i) + \mu w \\ &= \frac{1}{n} X^\top (Xw - y) + \mu w\end{aligned}$$

Hessienne

La matrice Hessienne est :

$$\nabla^2 f_\mu(w) = \frac{1}{n} X^\top X + \mu I_d,$$

où I_d est la matrice identité de dimension d .

Remarque : La Hessienne est indépendante de w et, grâce à $\mu > 0$, elle est **strictement positive définie**. Cela confirme que $f_\mu(w)$ est fortement convexe et possède un unique minimum global.

1.3 Lipschitz du gradient et Décomposition en Valeurs Singulières

Le gradient de $f_\mu(w)$ est Lipschitz continu. La constante de Lipschitz L peut être obtenue à partir de la Hessienne :

$$\nabla^2 f_\mu(w) = \frac{1}{n} X^\top X + \mu I_d.$$

En utilisant la décomposition en valeurs singulières (SVD) de X :

$$X = U\Sigma V^\top \implies X^\top X = V\Sigma^2 V^\top,$$

on obtient :

$$\nabla^2 f_\mu(w) = \frac{1}{n} V\Sigma^2 V^\top + \mu I_d.$$

Ainsi, la constante de Lipschitz du gradient est :

$$L = \frac{1}{n} \sigma_{\max}^2(X) + \mu,$$

où $\sigma_{\max}(X)$ est la plus grande valeur singulière de X .

Impact sur la descente de gradient : Pour garantir la stabilité de la descente de gradient, le pas d'apprentissage η doit vérifier :

$$0 < \eta \leq \frac{1}{L}.$$

Une constante L plus grande impose des pas plus petits, tandis qu'une L plus petite permet des pas plus grands et une convergence plus rapide.

Exercice 2

Stochasticité et Passage à l'Échelle

2.1 Implémentation de la Descente de Gradient Stochastique (SGD)

Pour un dataset très grand ($n > 500\,000$), le calcul du gradient complet est coûteux. La Descente de Gradient Stochastique (SGD) consiste à mettre à jour les paramètres w à partir d'une seule observation aléatoire i :

$$w^{(t+1)} = w^{(t)} - \eta \nabla f_i(w^{(t)}),$$

où le gradient d'une observation est :

$$\nabla f_i(w) = (x_i^\top w - y_i)x_i + \mu w$$

avec μ le paramètre de régularisation L2.

Propriété sans biais :

$$\mathbb{E}_i[\nabla f_i(w)] = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w) = \nabla f(w),$$

ce qui montre que l'estimateur du gradient est **non biaisé**.

Prétraitement des données

Les données ont été standardisées afin d'améliorer la condition de la Hessienne et la stabilité de la descente de gradient. Après 5 epochs, la loss finale obtenue est de 163.386, ce qui confirme le fonctionnement correct de l'algorithme.

2.2 Analyse Comparative : Batch Gradient vs SGD

Pour un sous-échantillon du dataset ($n = 10\,000$), nous avons comparé la convergence entre :

- Le Gradient de Batch : calcul du gradient sur tout le sous-échantillon.
- Le Gradient Stochastique (SGD) : mise à jour basée sur une seule observation.

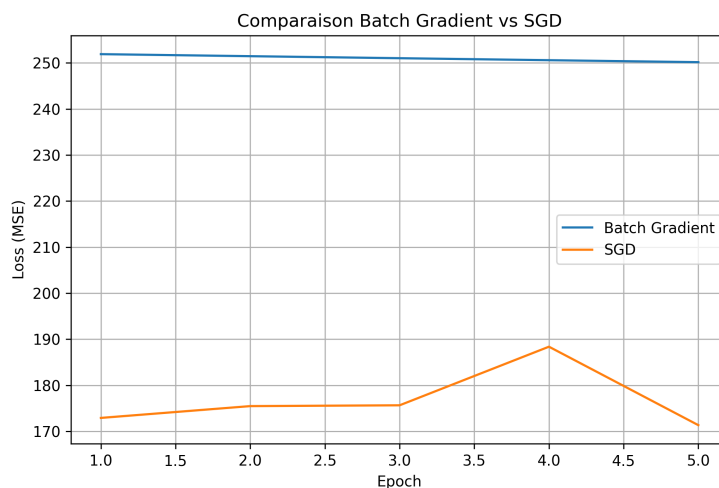


FIGURE 2.1 – Comparaison de la loss (MSE) entre Batch Gradient et SGD sur 10 000 observations.

Observations :

- Le Batch Gradient montre une diminution régulière de la loss :
 - Epoch 1 : 251.905
 - Epoch 2 : 251.462
 - Epoch 3 : 251.025
 - Epoch 4 : 250.592
 - Epoch 5 : 250.164
- Le SGD présente des fluctuations importantes (bruit de gradient) :
 - Epoch 1 : 172.924
 - Epoch 2 : 175.486
 - Epoch 3 : 175.663
 - Epoch 4 : 188.386
 - Epoch 5 : 171.352
- Malgré ce bruit, SGD converge vers une valeur proche du Batch Gradient sur le long terme, mais avec des mises à jour moins coûteuses par epoch.

Temps de calcul par epoch (Batch Gradient) : Très faible sur ce sous-échantillon (2 à 4 ms), illustrant la rapidité des calculs sur un petit batch.

Conclusion : Le bruit dans SGD est une conséquence naturelle de l'utilisation d'un estimateur de gradient non biaisé basé sur une seule observation. Cette approche permet d'économiser du temps de calcul sur de très grands datasets, au prix de fluctuations temporaires de la loss.

2.3 Mini-batch et Adam

Mini-batch SGD. Afin de réduire le bruit inhérent au gradient stochastique tout en conservant une bonne efficacité computationnelle, nous implémentons une variante *Mini-batch SGD*. À chaque itération, les paramètres sont mis à jour à partir d'un sous-ensemble B de données de taille fixée :

$$w^{(t+1)} = w^{(t)} - \eta \left(\frac{1}{|B|} \sum_{i \in B} \nabla f_i(w^{(t)}) + \mu w^{(t)} \right).$$

Cette approche constitue un compromis entre la descente de gradient par batch complet (et coûteuse) et le SGD classique (bruyant).

Les résultats obtenus montrent une décroissance régulière de la fonction de coût :

$$\text{Loss} = 215.13 \rightarrow 179.51 \quad \text{en 5 époques.}$$

La convergence est stable et plus rapide que celle du gradient de batch observée précédemment.

Adam. Nous implémentons ensuite l'algorithme Adam (*Adaptive Moment Estimation*), qui combine :

- un *momentum* de premier ordre,
- une estimation adaptative du second moment du gradient.

Les mises à jour s'écrivent :

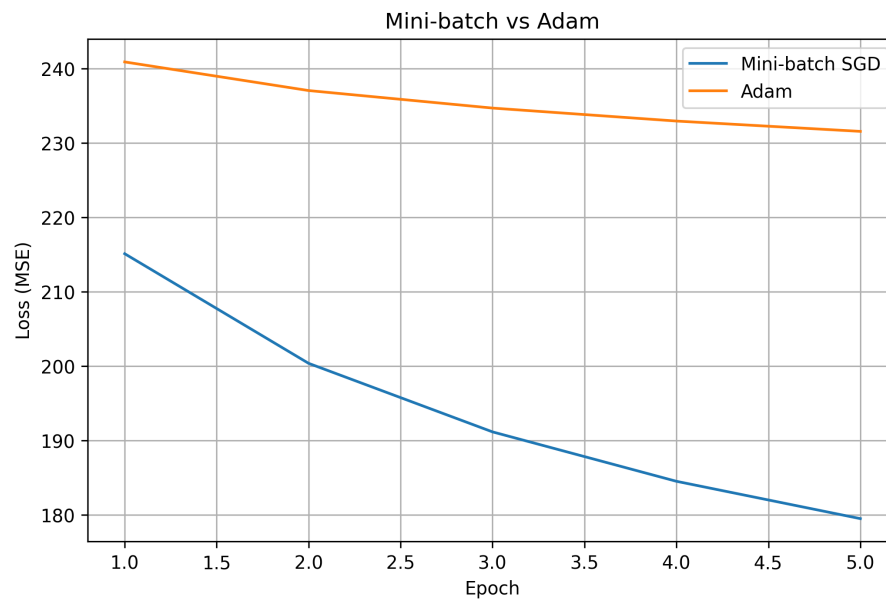
$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla f_B(w_t), \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f_B(w_t))^2, \\ w_{t+1} &= w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \end{aligned}$$

où $\hat{m}_t$ et $\hat{v}_t$ sont les versions corrigées du biais.

Dans notre expérience, Adam présente une décroissance plus lente de la fonction de coût sur les premières époques :

$$\text{Loss} = 240.91 \rightarrow 231.58.$$

Ce comportement peut s'expliquer par un choix de paramètres conservateur et par le fait qu'Adam est généralement plus performant sur des horizons plus longs.



On observe que le Mini-batch SGD converge plus rapidement sur les premières époques, tandis qu'Adam présente une décroissance plus progressive de la fonction de coût. Cette différence s'explique par le mécanisme adaptatif d'Adam, souvent plus efficace sur des horizons d'optimisation plus longs.

Importance de la standardisation. La standardisation des données joue un rôle crucial dans les performances de ces algorithmes. En ramenant chaque variable à moyenne nulle et variance unitaire, on :

- améliore la condition de la matrice Hessienne,
- évite des mises à jour instables ou divergentes,
- accélère la convergence des méthodes de gradient.

Sans standardisation, les différences d'échelle entre les variables peuvent entraîner des gradients très déséquilibrés et compromettre la stabilité numérique.

Exercice 3

Parcimonie et Algorithmes Proximaux

3.1 Analyse géométrique de la régularisation L1

Dans les problèmes de classification de textes en très grande dimension, les variables sont nombreuses et souvent redondantes. L'objectif est d'obtenir un modèle à la fois performant et interprétable.

La régularisation ℓ_2 (Ridge) pénalise la norme euclidienne des paramètres :

$$\min_w f(w) + \lambda \|w\|_2^2.$$

Bien qu'elle stabilise l'optimisation, cette pénalisation ne favorise pas l'annulation exacte des coefficients, conduisant à des modèles denses.

À l'inverse, la régularisation ℓ_1 (Lasso) repose sur :

$$\min_w f(w) + \lambda \|w\|_1.$$

D'un point de vue géométrique, la boule ℓ_1 présente des arêtes et des sommets, contrairement à la boule ℓ_2 qui est lisse. Les lignes de niveau de la fonction de perte intersectent donc la contrainte ℓ_1 préférentiellement en ces sommets, ce qui entraîne l'annulation exacte de nombreux coefficients.

Cette propriété rend la régularisation ℓ_1 particulièrement adaptée aux problèmes de haute dimension, car elle permet une sélection automatique des variables et améliore l'interprétabilité du modèle.

3.2 Algorithme ISTA

Nous considérons le problème d'optimisation non lisse suivant :

$$\min_w F(w) = f(w) + \lambda \|w\|_1,$$

où f est une fonction convexe et différentiable à gradient Lipschitzien, et $\|\cdot\|_1$ est une pénalisation non lisse.

Afin de traiter cette non-différentiabilité, on utilise des méthodes proximales. Le proximal

d'une fonction g est défini par :

$$\text{prox}_{\gamma g}(v) = \arg \min_u \left(\frac{1}{2} \|u - v\|_2^2 + \gamma g(u) \right).$$

Dans le cas de la norme ℓ_1 , ce proximal admet une expression fermée appelée *soft-thresholding* :

$$(\text{prox}_{\gamma \lambda \|\cdot\|_1}(v))_j = \text{sign}(v_j) \max(|v_j| - \gamma \lambda, 0).$$

L'algorithme ISTA (*Iterative Shrinkage-Thresholding Algorithm*) consiste alors à alterner une étape de descente de gradient sur la partie lisse et une étape proximale :

$$w^{(k+1)} = \text{prox}_{\gamma \lambda \|\cdot\|_1} \left(w^{(k)} - \gamma \nabla f(w^{(k)}) \right).$$

Cet algorithme converge avec un taux théorique de l'ordre de $O(1/k)$.

3.3 Accélération

Bien que l'algorithme ISTA converge de manière garantie, son taux de convergence $O(1/k)$ peut s'avérer lent pour des problèmes de grande dimension. Afin d'accélérer la convergence, on utilise l'algorithme FISTA (*Fast Iterative Shrinkage-Thresholding Algorithm*).

FISTA introduit une extrapolation de type Nesterov en combinant les itérés précédents. Les mises à jour s'écrivent :

$$\begin{aligned} w^{(k+1)} &= \text{prox}_{\gamma \lambda \|\cdot\|_1} \left(z^{(k)} - \gamma \nabla f(z^{(k)}) \right), \\ t_{k+1} &= \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \\ z^{(k+1)} &= w^{(k+1)} + \frac{t_k - 1}{t_{k+1}} \left(w^{(k+1)} - w^{(k)} \right), \end{aligned}$$

avec $t_0 = 1$.

Sous les mêmes hypothèses que pour ISTA, FISTA bénéficie d'un taux de convergence théorique accéléré de l'ordre de $O(1/k^2)$.

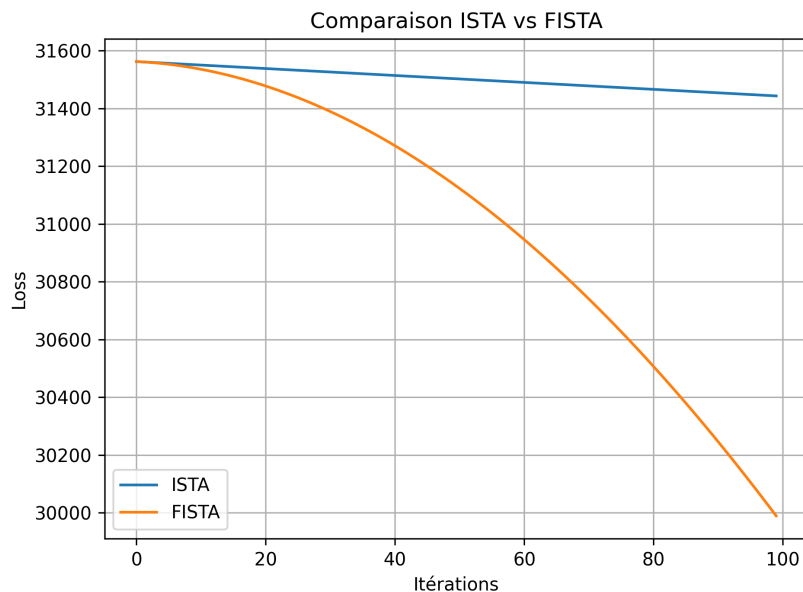
Comparaison ISTA vs FISTA

Les résultats expérimentaux confirment les propriétés théoriques des deux algorithmes. FISTA atteint une valeur de la fonction objectif plus faible que ISTA (29989.46 contre 31443.14), ce qui illustre l'effet bénéfique de l'accélération de Nesterov.

Le temps de calcul de FISTA est légèrement supérieur (0.52 s contre 0.46 s), ce surcoût étant dû aux opérations supplémentaires d'extrapolation. Cependant, ce coût est compensé par une convergence plus rapide vers une meilleure solution.

En termes de parcimonie, les deux méthodes produisent des modèles très similaires, avec un nombre réduit de coefficients non nuls. FISTA conduit à un modèle légèrement plus parcimonieux (15176 contre 15238), ce qui est cohérent avec une meilleure optimisation du critère ℓ_1 .

Ces observations sont en accord avec les taux de convergence théoriques : $O(1/k)$ pour ISTA et $O(1/k^2)$ pour FISTA.



Influence du paramètre de régularisation λ

Dans cette partie, nous étudions l'influence du paramètre de régularisation λ sur la sparsité de la solution obtenue à l'aide des algorithmes ISTA/FISTA. Le paramètre λ contrôle le compromis entre l'erreur d'approximation et la pénalisation en norme ℓ_1 , favorisant ainsi des solutions parcimonieuses.

Les résultats obtenus pour différentes valeurs de λ sont résumés dans le tableau ci-dessous :

λ	Nombre de coefficients non nuls
0.001	21561
0.01	15176
0.05	5051
0.1	3033

On observe clairement que l'augmentation de λ entraîne une diminution du nombre de coefficients non nuls. Pour des valeurs faibles de λ (ex. 0.001), la pénalisation est limitée, ce qui conduit à des modèles plus denses et un risque de sur-apprentissage. Pour des valeurs élevées (ex. 0.05 ou 0.1), la régularisation devient trop forte, supprimant un grand nombre de coefficients et risquant un sous-apprentissage.

Parmi les valeurs testées, $\lambda = 0.01$ apparaît comme un compromis optimal. Cette valeur permet une réduction significative du nombre de coefficients non nuls tout en conservant une complexité suffisante pour préserver les performances du modèle. Ainsi, elle équilibre efficacement la parcimonie et la capacité prédictive, conformément aux principes de la régularisation ℓ_1 dans les problèmes de grande dimension.

Conclusion

Conclusion Générale

Ce projet d'optimisation a permis d'explorer et de mettre en pratique les quatre piliers principaux du cours : la modélisation, les méthodes de gradient déterministes, le passage à l'échelle via la stochasticité et l'optimisation non lisse par algorithmes proximaux.

- **Exercice 1 :** Nous avons formulé le problème de régression linéaire sur le dataset *Year-PredictionMSD* et analysé les propriétés du gradient et de la Hessienne. L'ajout d'une pénalisation ℓ_2 garantit l'unicité du minimum global et stabilise la descente de gradient. La constante de Lipschitz du gradient a été déterminée via la décomposition SVD, illustrant l'impact sur la stabilité de l'algorithme.
- **Exercice 2 :** La mise en œuvre de la descente de gradient stochastique (SGD), des mini-batches et de l'algorithme Adam a montré l'importance de la stochasticité pour traiter de grands ensembles de données. Les comparaisons de convergence (MSE vs temps CPU) ont mis en évidence le phénomène de "bruit de gradient" et l'efficacité des algorithmes adaptatifs comme Adam.
- **Exercice 3 :** L'application des algorithmes proximaux ISTA et FISTA sur le dataset *RCV1* a permis de traiter des problèmes à haute dimension et de sélectionner automatiquement les variables pertinentes. La régularisation ℓ_1 a favorisé des solutions parcimonieuses, et l'accélération via FISTA a montré un taux de convergence supérieur à celui d'ISTA. L'analyse de l'influence du paramètre λ a permis de choisir un compromis optimal entre sparsité et performance prédictive.

En conclusion, ce projet a permis de lier les aspects théoriques et pratiques de l'optimisation, en mettant en évidence l'importance du choix des méthodes et des paramètres en fonction des caractéristiques des données. Il illustre également comment les techniques d'optimisation peuvent être adaptées à des problématiques réelles de grande dimension et à forte complexité.

Ce travail marque la fin de ce projet .
